



Data Visualization

Maximum intensity projection at warp speed

Lukas Mroz*, Andreas König, Eduard Gröller

*Institute of Computer Graphics, Vienna University of Technology, 5th Floor, Favoritenstrasse 9/5 Stock, Karlsplatz 13/186/2,
A-1040 Vienna, Austria*

Abstract

Maximum intensity projection (MIP) is a volume rendering technique which is used to extract high-intensity structures from volumetric scalar data. At each pixel the highest data value encountered along the corresponding viewing ray is determined. MIP is commonly used to extract vascular structures from medical MRI data sets, i.e., angiography. The usual way to compensate for the loss of spatial and occlusion information in MIP images is to view the data from different view points by rotating them. As the generation of a MIP is usually non-interactive, this is done by calculating multiple images offline and playing them back as an animation. In this paper a novel algorithm is proposed which is capable of interactively generating Maximum Intensity Projection images even on low-end hardware using parallel projection. Two methods for preprocessing data and removing voxels which will due to their neighborhood never contribute to a MIP are discussed. The remaining voxels are stored in a way which guarantees optimal cache coherency regardless of the viewing direction. For use on low-end hardware, a preview-mode is included which renders only the more significant parts of the volume during user interaction. Furthermore, we demonstrate the usability of our data structure for extensions of the MIP technique like *MIP with depth-shading* and *local maximum intensity projection (LMIP)*. © 2000 Elsevier Science Ltd. All rights reserved.

Keywords: Volume visualization; Maximum intensity projection; Real-time rendering; Angiography

1. Introduction

The ability to depict blood vessels is of enormous importance for many medical imaging applications. CT and MRI scanners are used to obtain volumetric data sets, which then allow the extraction of vascular structures. Especially data originating from MRI, which are most frequently used for this purpose, exhibit some properties which make the application of standard volume visualization techniques like ray casting [1] or iso-surface extraction [2] difficult. MRI data sets usually contain a significant amount of noise. Inhomogeneities in the sampled data make it difficult to extract surfaces of objects by specifying a single iso-value.

MIP exploits the fact, that within angiography data sets the data values of vascular structures are higher than the values of the surrounding tissue. By depicting the maximum data value seen through each pixel, the structure of the vessels contained in the data is captured. A straightforward method for calculating MIP is to perform ray casting and search for the maximum sample value along each ray instead of the usual compositing process done in volume rendering. In contrast to direct volume rendering, no early ray termination is possible and the whole volume has to be processed. Depending on the quality requirements of the resulting image, different strategies for finding the maximum value along a ray can be used.

- *Analytical solution:* for each data cell which is intersected by the ray the maximum value encountered by the ray is calculated analytically. Trilinear blending of the cell vertices is usually assumed for data values within a cell, thus the maximum of a third degree polynomial has to be calculated. This is the most

*Corresponding author. Tel.: +43-1-58801-18683; fax: +43-1-58801-18698.

E-mail addresses: mroz@cg.tuwien.ac.at (L. Mroz),
koenig@cg.tuwien.ac.at (A. König), groeller@cg.tuwien.ac.at
(E. Gröller).

accurate but also computationally most expensive method [3].

- *Sampling and interpolation*: as usually done for ray casting, data values are sampled along a ray using trilinear interpolation. The cost of this approach depends on how many interpolations can be avoided that will not affect the ray maximum [3,4].
- *Nearest-neighbor interpolation*: values of data points closest to the ray are used as maximum estimations. In combination with discrete ray traversal this is the fastest method. As no interpolation is performed, the voxel structure is visible in the resulting image as aliasing [5].

Recent algorithms for MIP employ a set of approaches for speeding up the rendering:

- *Ray traversal and interpolation optimization*: Sakas et al. [3] interpolate only if the maximum value of the examined cell is larger than the ray-maximum calculated so far. For additional speedup they use integer arithmetic for ray traversal and a cache-coherent volume storage scheme. Zuiderveld et al. [4] apply a similar technique to avoid trilinear interpolations. In addition, cells containing only background noise are not interpolated. For further speedup a low-resolution image containing lower-bound estimations for the maximum of clusters of rays is generated before the projection. Cells with maximum values below this bound can be skipped when the final image is generated. Finally, a distance-volume is used to skip empty spaces efficiently.
- *Using graphics hardware*: Heidrich et al. [6] use conventional polygon rendering hardware to simulate MIP. Several iso-surfaces for different threshold values are extracted from the data set. Before rendering, the geometry is transformed in a way, that the depth of a polygon corresponds to the data value of its iso-surface. MIP is approximated by displaying the z-buffer as a range of gray values. Recently, the VolumePro board [7] became available as a purely hardware-based solution for MIP. The board is able to produce medium-quality MIP using shear-warp-based projection at interactive frame rates.
- *Splatting and shear warp*: Several approaches [5,8] exploit the advantages of shear-warp rendering [9] to speed up MIP. Cai et al. [5] use an intermediate “worksheet” for compositing interpolated intensity contributions for projection of a single slice of the volume. The worksheet is then combined with the shear image to obtain the maxima. Several splatting modes with different speed/quality tradeoffs are available, run-length encoding and sorting of the encoded segments by value are used to achieve further speedup.

As a MIP image contains no shading information, depth and occlusion information is lost (see Fig. 1a). Structures with higher data values lying behind a lower

valued object appear to be in front of it. The most common way to ease the interpretation of such images is to animate the viewpoint while viewing. Another approach is to modulate the data values by their depth to achieve a kind of depth shading [6] (see Fig. 1b). As the data values are modified before finding the maximum, MIP and depth shaded MIP of the same data may display different objects.

Depth shading provides some hints on the spatial relation of objects. However, its performance is rather poor especially for tightly grouped structures. In such cases Closest Vessel Projection [3] or LMIP [10] can be used. Similar to MIP, for LMIP the volume is traversed along a ray, but the first local maximum which is above a user defined threshold is selected as the pixel color. If no value above the threshold is found along a ray, the global maximum along the ray is used for the pixel. With a high threshold value this method produces the same result as MIP, with a carefully selected one, less intense structures in front of more intense background are depicted, producing an effect similar to shading (see Figs. 1c and d). As this method is very sensitive to the setting of the threshold, the ability to interactively tune this parameter is extremely important.

In the following, we present a novel approach to generate MIP images really fast. In Section 2 two algorithms for preprocessing the volume data to eliminate voxels, which will not contribute to a MIP, are discussed. Furthermore, we present a volume storage scheme which gets rid of overhead associated with skipping those voxels for rendering. In Section 3 algorithms for generating MIP using either template-based projection or shear-warp projection are presented. In Section 4 extensions of the algorithm for generating depth-shaded MIP and LMIP using our data structure are discussed.

2. Preprocessing and volume storage

Most approaches to optimize volume traversal aim at excluding voxels from the traversal and rendering process, which contain less-important information like low-valued background noise. In fact, in addition to this low-importance data, volumetric data sets usually contain a remarkable amount of voxels which never contribute to a MIP image. A voxel V does never contribute to a MIP and can be discarded if all possible rays through the voxel hit another voxel W with $d(W) \geq d(V)$ either before or after passing through V , where $d(V)$ is the data value at voxel v . This fact can be exploited when original voxel values are used for rendering using nearest-neighbor interpolation.

In the following sections, two algorithms for identifying non-contributing voxels are presented. The first approach performs the classification based on the local neighborhood of a voxel, identifying voxels invisible from

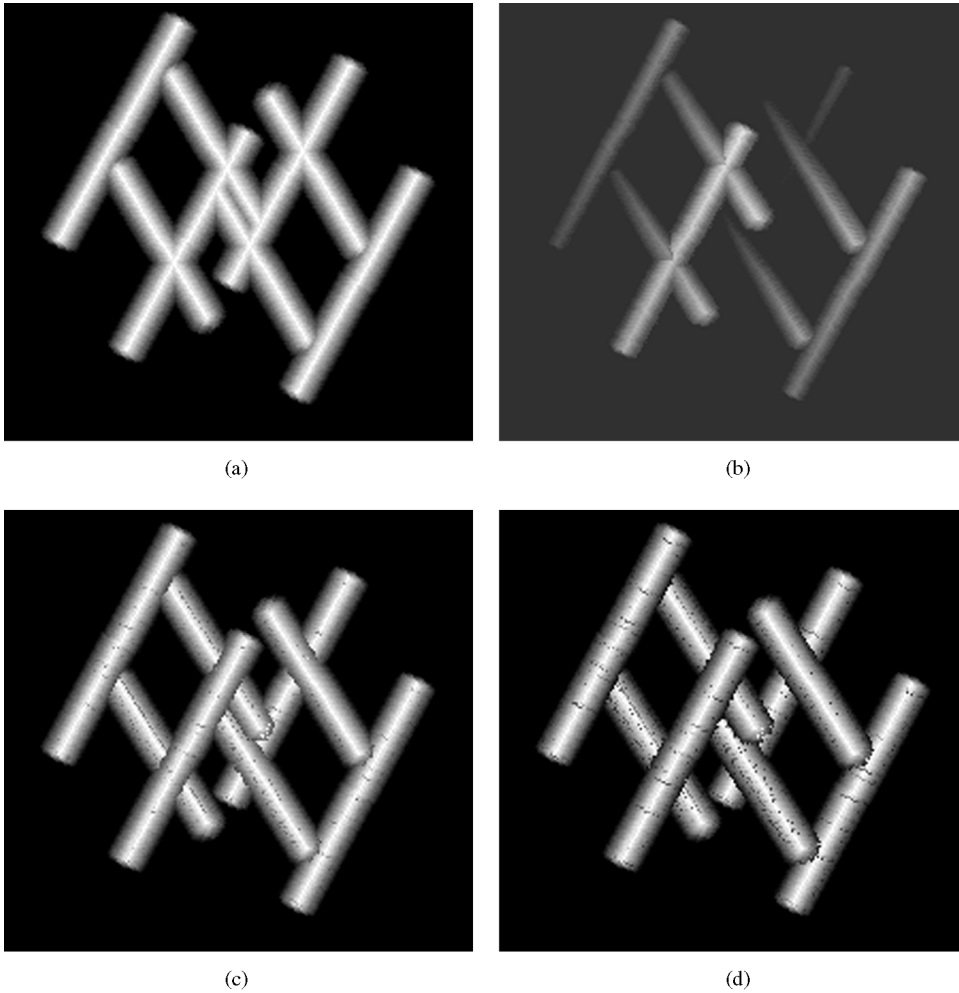


Fig. 1. (a) MIP of a test data set containing soft bounded cylinders; (b) although depth shading provides some depth impression, the cylinders seem to intersect; (c), (d) while LMIP provides more information on the spatial structure, the results are sensitive to the threshold: (c) high, (d) low.

any viewing direction. The second algorithm groups possible viewing directions into several clusters and produces a set of potentially contributing voxels for each cluster. The view-point-dependent elimination achieves much better elimination rates at the cost of storing several sets of voxels for rendering.

For the elimination and rendering process voxels are treated as cells with a constant data value. A viewing ray consists of a 26-connected sequence of voxels, comparable to one produced by a Bresenham line algorithm.

2.1. Neighborhood-based elimination

A simple approach for the identification of “hidden” voxels is to examine direct neighbors W_i of a voxel V . If

V does not influence the maximum of any ray passing through V and any of its neighbors W_i , it can be removed. Obviously this is the case if all rays through V pass through a neighbor W_i with $d(W_i) \geq d(V)$ either before or after passing through V . This condition can be tested by examining the values of all 26 direct neighbors W of V :

- If for all i $d(W_i) \geq d(V)$, all rays through W_i and V will have at least the value $d(W_i)$ and are not influenced by V (see Fig. 2a).
- If for some i $d(W_i) < d(V)$, but all rays through W_i and V pass through another neighbor W_j of V with $d(W_j) \geq d(V)$, the ray values will at least be $d(W_j)$. Thus V has also no influence on rays through W_i (see Fig. 2b).

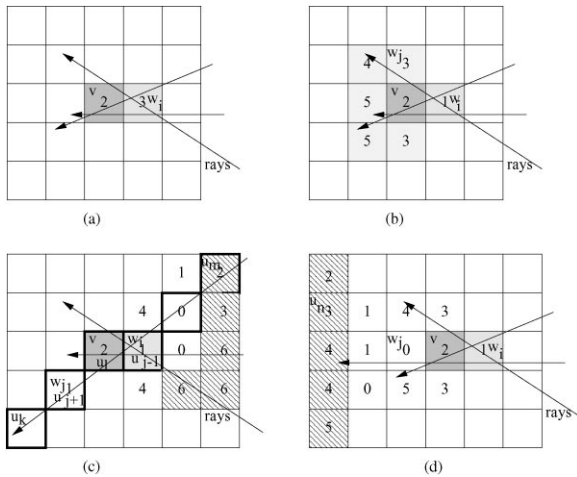


Fig. 2. Detection of voxels V which do not contribute to MIP. For simplicity, the different cases are shown in 2D. (a) V irrelevant for rays through W_i as $d(W_i) \geq d(V)$ (b) V irrelevant for rays through W_j . (c) V irrelevant for rays through W_i as ray maxima are determined by voxels U_m . (d) V irrelevant for rays through W_i , as ray maxima are determined by U_n .

Voxel elimination which is based on these two tests can be performed rapidly (see Table 1) and requires just access to direct neighbors of each voxel. If the above tests fail for a group of rays through W_i, V , and W_j , with $d(W_i) < d(V)$ and $d(W_j) < d(V)$, the influence of a larger neighborhood of V on the value of these rays can be checked. Each ray passes through a sequence of voxels U_k with $U_l = V$, $U_{l-1} = W_i$ and $U_{l+1} = W_j$ as the volume is traversed. If each of those rays passes through some voxel U_m with $m < l - 1$ and $d(U_m) > d(V)$, V has no influence on the values of the rays through W_i and W_j (Fig. 2c). Similarly, if all rays through W_i and W_j pass through voxels U_n with $n > l + 1$ and $d(U_n) > d(V)$, V has no influence on the values of those rays (Fig. 2d).

The check of the influence of more distant volume areas can be realized by recursively propagating bundles of rays starting at direct neighbors of V (see Table 1). If

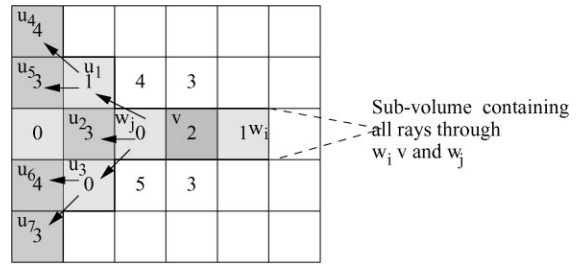


Fig. 3. Tracking the influence of distant parts of the volume on rays through W_i, V and W_j . As all of them pass either through U_1, U_2 or U_3 and $d(U_2) > d(V)$, V does not influence rays through U_2, W_j, V and W_i . As $d(U_1) < d(V)$ and $d(U_3) < d(V)$, further tracking of rays passing through U_1 or U_3 is required.

a voxel U is hit by a bundle of rays and $d(U) \geq d(V)$ the tracking of the bundle is terminated, as V has no influence on the rays. If $d(U) < d(V)$ the bundle is split and tracked in a subset of the neighbors of U (Fig. 3).

To improve efficiency and avoid unnecessary recursive checks, values of voxels scheduled for removal are replaced by the minimum of their obscuring values (which is always larger or equal to their original value). To compensate for noise in the data and to increase the number of rejected voxels a user specified tolerance value ε can be included in the comparison process, which artificially lowers the value of each checked voxel V by ε . For voxel reduction rates using different elimination efforts and tolerance values ε please refer to Table 1. Although recursive elimination requires significantly more time, it may be the method of choice if data is prepared on a high-end machine for viewing with low-end hardware over a network.

2.2. Shadow sweep elimination

Although the approach in Section 2.1 is able to remove approximately half of a dataset, many voxels have to remain in the volume even if they only contribute to a MIP for a narrow range of viewing directions. If enough memory is available for redundant storage of the data, more efficient preprocessing can be performed by

Table 1

Volume reduction: Fast optimization (“Fast”) considers direct neighbors, full optimization (“Full”) performs recursion up to a distance of 10 voxels. Direction-dependent optimization (“dir.dep”) produces 24 view-dependent sets of voxels. The results show the percentage of voxels removed and the time required for preprocessing on a PII/450 PC

Data set	Size	Fast ε 0%	Full ε 0%	Fast ε 1%	Full ε 1%	Dir.dep.
mr_ang.	$256^2 \times 64$	25%/5 s	47%/270 s	29%/5 s	53%/270 s	73%/72 s
mr01	$256^2 \times 74$	33%/6 s	46%/208 s	45%/6 s	55%/208 s	72%/76 s
mr03	$256^2 \times 124$	29%/10 s	58%/562 s	29%/10 s	58%/562 s	70%/120 s

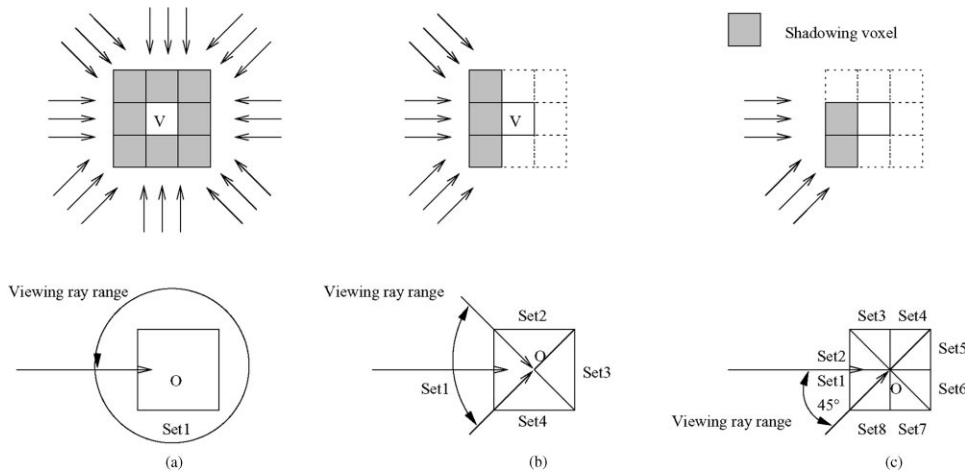


Fig. 4. Removal of hidden voxels in 2D: (a) viewpoint-independent preprocessing: relevance of voxel V depends on rays passing through eight neighbors. (b) 4 view-sets: relevance of V depends on three neighbors. (c) 8 view-sets: dependency minimized (two neighbors).

subdividing all possible viewing directions into disjoint view-sets in a way which minimizes dependencies among voxels (Fig. 4). The highest efficiency of removal in 3D can be achieved if 24 such view-sets are used. For each view-set a set of potentially visible voxels is computed using a two-pass procedure:

- Shadowed voxel sweep:** Mark all voxels V in shadow of (local) maxima as non-contributing. This is the case when $d(V)$ is lower than the smallest ray maximum of rays through V collected before reaching V . This can be easily performed by sweeping through the volume and processing voxels in the order in which they are hit by an advancing front of rays. During this process a front is propagated which contains the smallest maximum values for rays entering each voxel directly ahead of the front. Considering the example in Fig. 5a, all viewing-rays of the given view-set through voxel V have first to pass either through V_1 or V_2 . The lowest possible values of rays at V_1 and V_2 respectively are $\min_1 = d(V_1)$ and $\min_2 = d(V_2)$, as both V_1 and V_2 are on the border of the volume. Thus the lowest possible value of a ray arriving at V is $\text{arr}_V = \min(\min_1, \min_2)$. As $d(V) > \text{arr}_V$, V may influence some of the arriving rays, and cannot be removed. The value of the rays after passing V equals $\max(\text{arr}_V, d(V))$ and influences the values of rays arriving at the successive voxels V_3 and V_4 .
- Leading voxel sweep:** During the second pass all low valued voxels in front of voxels containing unremoved (local) maxima are removed. The removal is performed in the same way as step 1 with the difference that the scan is performed opposite to the direction of the viewing rays (Fig. 5b). Note, that voxels removed dur-

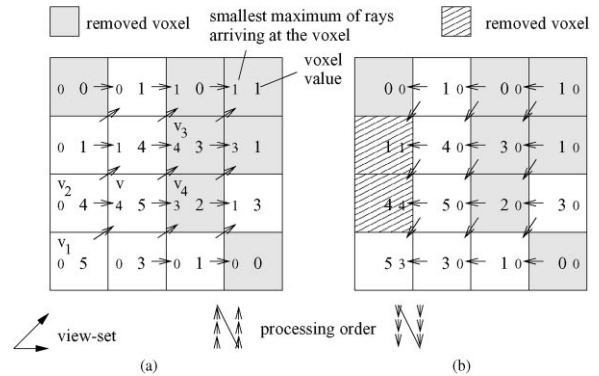


Fig. 5. View-point-dependent voxel removal (2D). (a) Shadowed voxel sweep. (b) Leading voxel sweep.

ing the shadowing sweep have no influence on ray maxima during the second sweep.

Direction-dependent preprocessing allows to remove about three quarters of all voxels for each view-set (see Table 1). During rendering, voxels of the view-set which contains the current viewing direction are selected and rendered.

2.3. Volume storage

Although distance volumes are an efficient way to skip large empty volume regions, they are less suited for volumes with large numbers of small empty spaces or even single empty voxels. In addition, several access operations to distance data stored in the empty space are usually required to skip a gap.

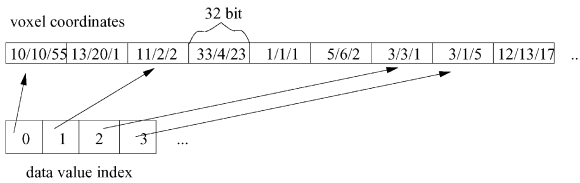


Fig. 6. Value-sorted array structure: All voxels are sorted according to data value and their position in space is stored in an array. An additional index points to the first voxel in the array for each possible data value.

The data structure commonly used to store volume data is a 3D array which explicitly stores data values and encodes their position in space implicitly in the value's position in storage.

A way to completely get rid of unwanted voxels from a volume is to store the interesting voxels in an array of voxel-positions and to encode the value of the voxels implicitly into their position in this value-sorted array (see Fig. 6). The required sorting of all voxels according to their data value can be performed in linear time, as the limited range of possible data values allows to use histogram-based sorting. The coordinates of a voxel within a volumetric dataset can be packed into a 32-bit integer, allowing the encoding of volumes of up to $2048 \times 2048 \times 1024$ voxels. A straightforward conversion of a 16-bit/value volume to a 32-bit position representation would double the memory cost. Omitting the voxels which have been marked by the preprocessing step as irrelevant leads to a factor of 0.8–1.5 in storage size compared to the original data for non-direction dependent data. For direction-dependent data, approximately one-quarter of the original data have to be stored for all

24 view-sets. The overall memory cost for direction dependent preprocessing is thus 12 times higher than the cost of the original volume.

By storing voxels in a value-sorted array we gain several important advantages for MIP:

- The voxels can be splatted in the order of ascending data values as the array is traversed. Therefore, *comparing* the value of the actual voxel with the screen content is *not necessary* at all. The value of the actual voxel is always larger or equal to the content of the screen, thus its projection can be written into the image. Omitting the comparison during projection results in 10% better performance.
- As the array is traversed in the same way independent of the viewing direction, *optimal cache coherency* is always achieved. The same code accessing data in a linear way is up to eight times faster than in the case of extremely misaligned access.
- As blood vessels are represented by high data values, lower data values usually contain less important information. If an interactive display of the full data set is not possible on the given hardware, lower intensity values can be simply skipped (see Fig. 7) to achieve interactivity during user interactions. The algorithm can be adjusted to display MIP at *an arbitrary frame rate*. The number of voxels which can be displayed at a specific frame rate can be easily derived automatically from a performance measurement of the number of voxels which can be rendered per second.
- Instead of directly mapping data values to a linear ramp of gray values for viewing ($d_{min} \Rightarrow g_{min}$, $d_{max} \Rightarrow g_{max}$), medical data sets are usually viewed using a “window” to improve contrast and to focus on certain details. The window is defined by a center value

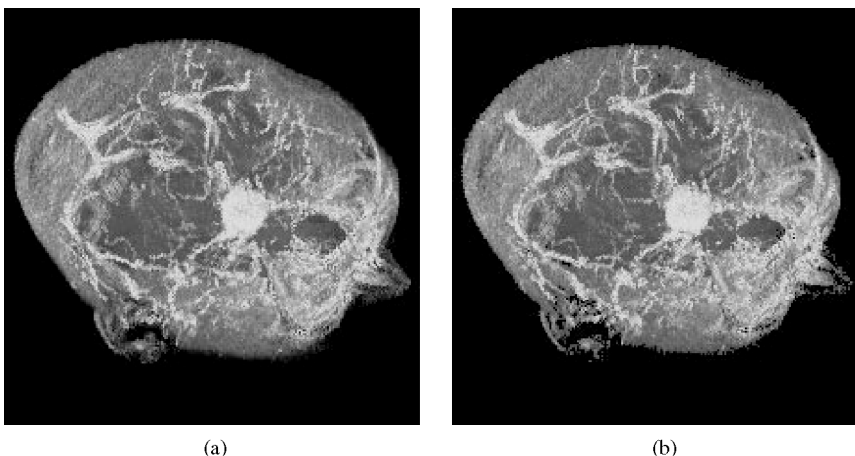


Fig. 7. (a) MIP of the mr01 data set, 2.6M voxels. (b) Interactive preview of the same data at 10fps on a P233MMX with approximately 25% of the volume data displayed. Only slight differences are noticeable.

c and a width w which maps all data values below $c - w/2$ to black, all data values above $c + w/2$ to white, and the data in between to a gray ramp. As realistic window functions as used by doctors map significant portions of the data to black, the corresponding parts of the value-sorted array can be skipped during rendering at almost zero cost. This results in an additional *speedup of up to 500%* compared to unwindowed data *under real-world conditions*.

The value-sorted array does not store neighborhood information. It is therefore most useful for algorithms which work with voxels rather than cells. Acquiring the eight data values of a cell from the array without access to the original data volume would be a rather inefficient task.

3. Maximum intensity projection

As the voxels of the volume are processed in an arbitrary spatial order which is defined by the order within the value-sorted array, fast incremental techniques for projecting voxels along a ray cannot be used. The parallel projection P of a point in 3D, however, can be expressed as the sum of independent projections of its x , y and z components:

$$\mathbf{P} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \mathbf{P} \begin{pmatrix} x \\ 0 \\ 0 \end{pmatrix} + \mathbf{P} \begin{pmatrix} 0 \\ y \\ 0 \end{pmatrix} + \mathbf{P} \begin{pmatrix} 0 \\ 0 \\ z \end{pmatrix}.$$

The projections of all possible voxel coordinates along a coordinate axis can be precalculated and stored into a template. A combination of three screen position templates $\mathbf{x}s$, $\mathbf{y}s$ and $\mathbf{z}s$, can be used to inexpensively calculate the position of each voxel's projection on the screen. For increased efficiency, the templates can immediately store offsets into the image buffer instead of $\mathbf{x}/\mathbf{y}$ screen coordinates. The resulting rendering loop is simple and efficient:

```
xs=make_template(viewmatrix,volumesize_x);
do the same for ys, zs;
for intensity = 0 to max do
  gray = window_function[intensity];
  if (gray! = BLACK)
    for index = first[intensity] to
      first[intensity + 1] - 1 do
      voxpos = value_sorted_array[index];
      screen[xs[voxpos.x] + ys[voxpos.y]
        + zs[voxpos.z]] = gray;
```

Similar to other MIP algorithms this approach is also a tradeoff between speed and quality. As each voxel of the data set is projected onto exactly one pixel, the size of the

resulting image depends on the size of the volume. Furthermore, the composition of multiple integer templates for determining the position of the projection in combination with the projection of a voxel onto exactly one pixel may produce “holes” within the projection of the volume when viewed from certain directions. Although clearly apparent in a static image, due to the high interactivity of this method these artifacts do not disturb the perception of the data during interactive work. The artifacts can be significantly reduced by using sub-pixel coordinates in x -direction. As the templates store offsets into the image buffer $(x + \text{img_width} * y)$ instead of $\mathbf{x}/\mathbf{y}$ coordinates, subsampling in y -direction is not possible.

To avoid holes in the image, shear-warp projection [9] can be used as a slightly slower alternative to template-based projection. Each voxel is projected to exactly one pixel of the shear-image, which is then warped to complete the projection.

4. Extensions

The algorithm presented in the previous section can be easily extended to generate depth-shaded images (Fig. 8a). Similar to the screen position templates, three z -templates are generated to calculate the depth of each voxel's projection. The intensity value of each voxel is modulated by it's depth and written into the image buffer only if the value of the pixel at this position is smaller than the modulated value. As the depicted maximum values do not directly correspond to data values, applying this method to preprocessed volume data may produce results slightly differing from projecting non-preprocessed data.

The second extension of the algorithm is capable of generating LMIP images providing the possibility to interactively adjust the threshold parameter. For generating a MIP image, the order of examining samples along a ray is not relevant. Straightforward LMIP requires the samples to be processed front to back along the viewing ray in order to find the first local maximum. Using z -templates for voxel-depth calculation and two z -buffers per pixel allows to extract the closest local maximum from samples arriving in arbitrary order. The $z[\text{pix}]$ buffer stores the depth of the currently visible voxel, while $zb[\text{pix}]$ stores a “back-clipping” distance for each pixel which is used to skip voxels belonging to maxima behind the currently closest one. As the voxels are processed in order of ascending data value, all voxels below the LMIP-threshold can be first projected using the simple and fast MIP algorithm without z -calculation and local minimum tracking. Among all voxels above the threshold which are processed later, closest local maxima have to be found. At the beginning, $z[\]$ is initialized to contain the maximum possible

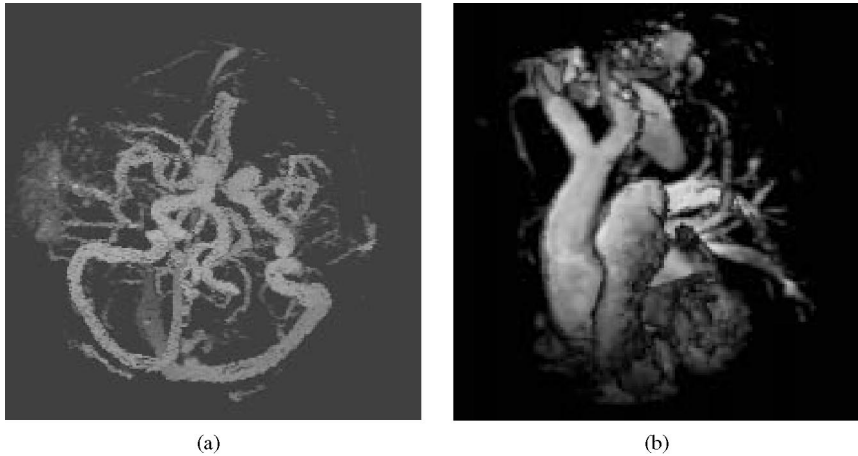


Fig. 8. (a) MIP of an angiography with depth shading. (b) LMIP of a stenosis of the aorta.

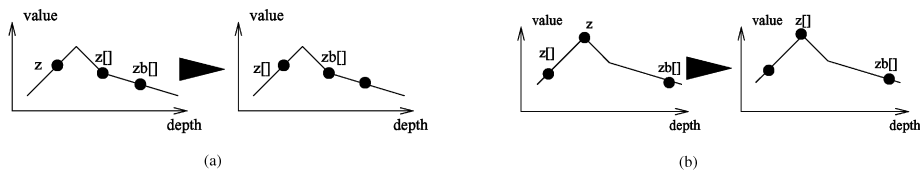


Fig. 9. Local maximum intensity projection for non-sequential samples: (a) new voxel in front of z (b) new voxel between z and z_b .

distance. With z and v containing the depth and value of a projected voxel, the closest local maximum along a ray is found by

```
if ( $z < z[\text{pix}]$ )
    screen[pix] =  $v$ ;  $z_b[\text{pix}] = z[\text{pix}]$ ;  $z[\text{pix}] = z$ ;
else if ( $z < z_b[\text{pix}]$ )
    screen[pix] =  $v$ ;  $z[\text{pix}] = z$ ;
```

The first condition detects voxels closer than the currently displayed voxel. As they have at least the same intensity as the currently displayed one, they are entered into the `screen` and `z[]` buffers, the back clipping plane is moved forward to the previous `z[]` position (Fig. 9a). The second condition takes care of new voxels which are placed between the currently displayed voxel and the back clipping distance and are at least of the same intensity as the current one. In this case the `screen` and `z[]` are set to the new voxel. Voxels behind the back clipping distance are ignored (Fig. 9b). As the value at `z_b[]` is always older than the value at `z[]` it is also smaller due to the ascending processing order of the voxels. Thus, any voxel which arrives later during processing and is located behind `z_b[]` can be ignored, as it does definitely not belong to the closest maximum.

5. Future work

Future work will concentrate on the use of cells instead of voxels in order to allow high-quality MIP rendering using the same value-sorted array. Apart from medical Imaging, the availability of interactive MIP may open interesting application fields to this rendering technique.

6. Results

The volume preprocessing technique as well as the MIP algorithms presented in this paper have been implemented as a Java applet (<http://www.cg.tuwien.ac.at/research/vis/vismed/RT-MIP/>). The rendering times in Table 2 have been measured on a PII/450 PC using the Java virtual machine of JDK 1.1.6 with a just in time compiler. The first value gives the time for rendering data preprocessed independent of the viewpoint, the second value is the time for rendering direction dependent preprocessed data. As timings for LMIP depend on the chosen threshold, measurements for the worst case are given. The window used for the measurements in the second column is $c = 2048/w = 2048$ for datasets 1 and 3 and $c = 2825/w = 2048$ for dataset 2. A C++ version

Table 2

Frame rendering times (with direction independent/direction dependent preprocessing)

Data set	MIP (ms)	MIP windowed (ms)	Depth-shaded (ms)	LMIP (ms)
mr_angio $256^2 \times 64$	169/94	27/21	209/118	< 305/180
mr01 $256^2 \times 74$	163/106	42/32	224/146	< 288/198
mr03 $256^2 \times 124$	254/219	13/11	338/267	< 455/370

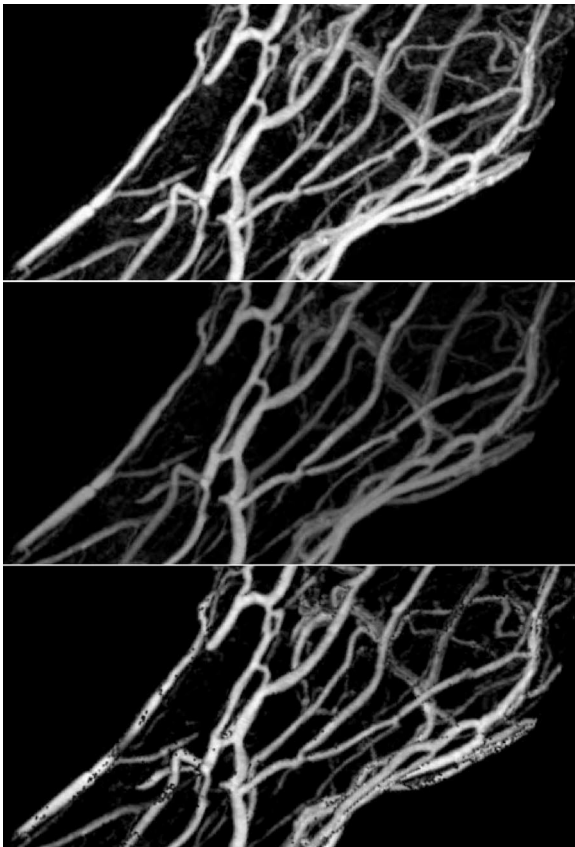


Fig. 10. MIP of the vessels in a hand: MIP, Depth-shaded MIP, LMIP.

of the MIP rendering algorithm has shown an approximately 30% better performance than the Java version.

Fig. 10 is provided for a visual comparison of the MIP algorithms discussed in this paper. It shows a $256 \times 256 \times 100$ angiography dataset of a hand rendered with MIP, DMIP and LMIP using our sorted array approach and shear-warp decomposition of the viewing matrix for projection.

7. Conclusion

We have presented a new rendering algorithm for maximum intensity projection (MIP) which runs at real-

time rates. In combination with a preprocessing step which removes parts of the volume which do not contribute to a MIP image and a volume storage scheme (value-sorted array) which eliminates overhead for traversing empty regions of the volume, the algorithm achieves highly interactive frame rates. Compared to other optimized MIP algorithms we gain up to an order of magnitude in speed, compared to a brute-force ray casting approach even two orders of magnitude. A preview mode for interactions on low-end hardware is provided basically “for free” by skipping voxels with low data values. Parallel projection is reduced to summing up three values stored in lookup tables. Two extensions of the algorithm allow to generate depth-shaded MIP and LMIP at comparable speed. Although some artifacts are present in the image as no interpolation and a quick but jaggy projection method is used, the high interactivity of this method makes it suitable for exploration of angiography data sets. Shear-warp projection may also be used as a slightly slower but higher quality projection method.

Acknowledgements

The work presented in this publication has been funded by the FWF as part of project P-12811/INF and by the VisMed project (<http://www.vismed.at>) which is supported by Tiani Medgraph, Vienna, <http://www.tiani.com>, and the FFF, Austria. The mr_angio, stenosis and hand data sets are courtesy of Tiani Medgraph GesmbH, Vienna. The mr01 and mr03 data sets are available from the United Medical and Dental Schools (UMDS) Image Processing Group in London. <http://www-igp.umds.ac.uk/archive/heads.html>

References

- [1] Kajiya JT. Ray tracing volume densities. Proceedings of ACM SIGGRAPH '84, 1984. p. 165–74.
- [2] Lorensen WE, Cline HE. Marching cubes: A high resolution 3D surface construction algorithm. In Proceedings of ACM SIGGRAPH '87, 1987. p. 163–89.

- [3] Sakas G, Grimm M, Savopoulos A. Optimized maximum intensity projection. Proceedings of the Fifth EUROGRAPHICS Workshop on Rendering Techniques, Dublin, Ireland, Italy, 1995. p. 55–63.
- [4] Zuiderveld KJ, Koning AHJ, Viergever MA. Techniques for speeding up high-quality perspective maximum intensity projection. *Pattern Recognition Letters* 1994;15:507–17.
- [5] Cai W, Sakas G. Maximum intensity projection using splatting in sheared object space. Proceedings EUROGRAPHICS '98, 1998. p. C113–24.
- [6] Heidrich W, McCool M, Stevens J. Interactive maximum projection volume rendering. Proceedings Visualization '95, 1995. p. 11–8.
- [7] Pfister H, Hardenbergh J, Knittel J, Lauer H, Seiler L. The volume pro real-time ray-casting system. Proceedings of ACM SIGGRAPH '99, 1999. p. 251–60.
- [8] Csebfalvi B, König A, Gröller E. Fast maximum intensity projection using binary shear-warp factorization. Proceedings of the Seventh International Conference in Central Europe on Computer Graphics, WSCG '99, 1999. p. 47–54.
- [9] Lacroute P, Levoy M. Fast volume rendering using a shear-warp factorisation of the viewing transform. Proceedings of ACM SIGGRAPH '94, 1994. p. 451–9.
- [10] Sato Y, Shiraga N, Nakajima S, Tamura S, Kikinis R. LMIP: Local maximum intensity projection — a new rendering method for vascular visualization. *Journal of Computer Assisted Tomography* 1998;22(6).